

Backend Engineering Challenge

Review Intelligence System

Estimated Time	2 - 3 hours	Difficulty	Easy
Stack	Python, Docker, Ollama	Deliverable	GitHub repo

1. Context

You are building a review analysis tool for a hotel. The hotel receives many customer reviews, but they are often long, repetitive, and difficult to analyze manually.

The hotel manager wants a fast way to understand what customers are actually saying: what guests liked most, what complaints appear repeatedly, and which topics matter most overall.

Your task: Build a system that processes a collection of hotel reviews and turns them into structured, deduplicated insights.

2. Requirements

2.1 Input

A .txt file where each line contains one hotel review.

2.2 Endpoint

When the /process_file endpoint is called, the system must:

1. Read all reviews from the input file.
2. For each review, use a small local LLM (llama3.2:1b via Ollama) to extract:
 - Key highlights (positives)
 - Key pain points (negatives)
3. Aggregate all extracted highlights into one list, and all pain points into another.
4. Generate embeddings for each extracted item.
5. Compare embeddings using cosine similarity.
6. For each list separately:
 - If multiple items have cosine similarity greater than 0.8, treat them as duplicates
 - Keep only one representative item from each group
 - Remove the rest
7. Return the final deduplicated list of highlights and pain points, each with a count of how many similar items it replaced.

2.3 Infrastructure

Everything must run with Docker Compose, including the app and the Ollama LLM server.

3. Example

3.1 Input File

```
The room was clean and the breakfast was excellent, but check-in took too long.
Amazing location near the beach. Staff were friendly, although the waiting time at check-in was frustrating.
Great breakfast and very clean rooms. The reception process was slow.
```

3.2 LLM Extraction Per Review

Review	Highlights	Pain Points
Review 1	Room was very clean; Breakfast was excellent	Check-in took too long
Review 2	Amazing location near the beach; Staff were friendly	Waiting time at reception was frustrating
Review 3	Great breakfast; Very clean facilities	Reception process was slow

3.3 Before Deduplication

Highlights	Pain Points
• Room was clean • Breakfast was excellent • Amazing location near the beach • Staff were friendly • Great breakfast • Very clean rooms	• Check-in took too long • Waiting time at check-in was frustrating • Reception process was slow

3.4 After Embedding + Cosine Similarity Deduplication

Obs: this can differ depending on the embedding model

Highlights (deduplicated)	Pain Points (deduplicated)
• Room was very clean (count: 2) • Breakfast was excellent (count: 2) • Amazing location near the beach (count: 1) • Staff were friendly (count: 1)	• Check-in took too long (count: 3)

4. Technical Requirements

4.1 Mandatory

- API with a /process_file POST endpoint
- Ollama running locally with llama3.2:1b for LLM extraction
- Sentence embeddings for semantic similarity
- Cosine similarity deduplication with threshold 0.8
- Docker Compose setup that spins up all services with a single command

- Clean, readable code with meaningful variable names and comments

4.2 Expected API Response Format

```
{
  "highlights": [
    { "item": "Room was very clean", "count": 2 },
    { "item": "Breakfast was excellent", "count": 2 },
    { "item": "Amazing location near the beach", "count": 1 },
    { "item": "Staff were friendly", "count": 1 }
  ],
  "pain_points": [
    { "item": "Check-in took too long", "count": 3 }
  ]
}
```

5. Deliverables

- A public GitHub repository containing all code
- A README with clear instructions on how to run the project locally using Docker Compose

Good luck! We look forward to seeing your solution.